Компьютерная графика

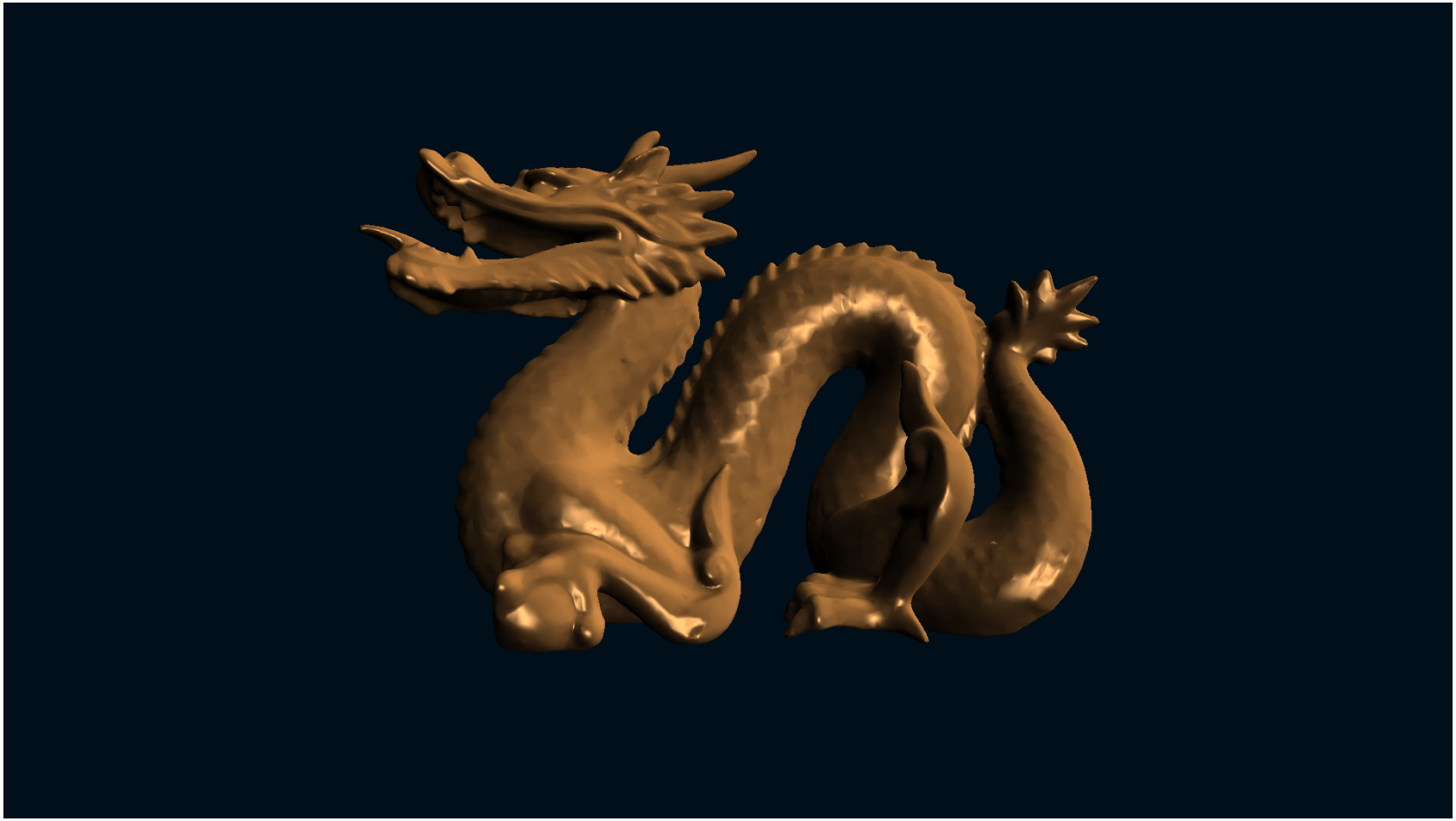
Практика 6: render to texture

2026

Практика 6

- Модель можно двигать и вращать стрелками

Практика 6



Задание 1

Создаём sampler, offscreen текстуру, view, и bind группу

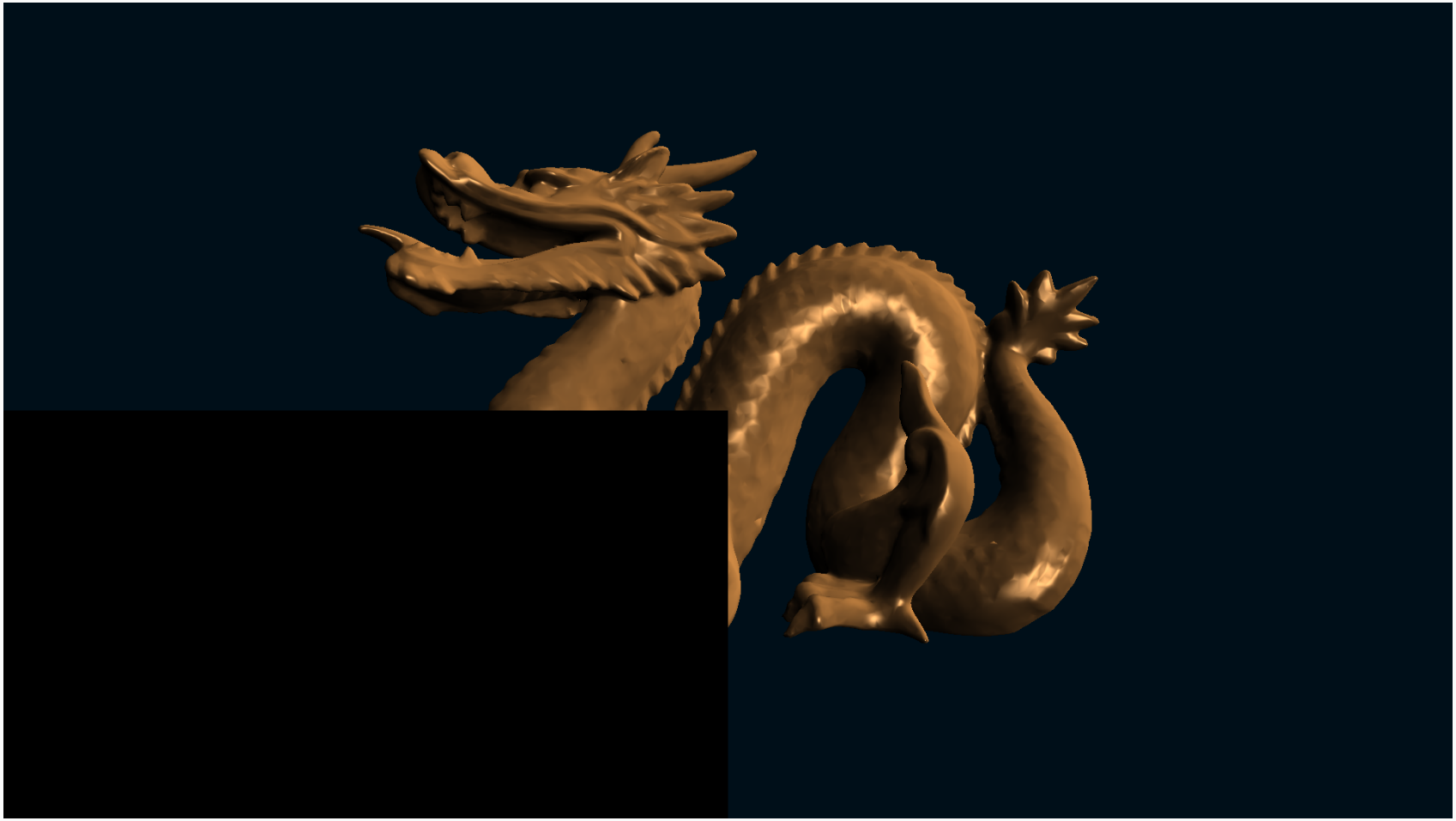
- Параметры sampler'а: `address_mode = ClampToEdge`, `*_filter = Linear`
- Формат текстуры – `RGBA8Unorm`, размер – $\frac{width}{2} \times \frac{height}{2}$
- Bind группа использует уже имеющийся в коде `bindGroupLayout`, в ней 2 entry: sampler и texture view
- Текстуру, view и bind группу нужно пересоздавать при изменении размеров экрана (так же, как буфер глубины)
- NB: Пока ничего не изменится, мы никак не используем эту текстуру :)

Задание 2

Рисуем созданную текстуру

- **После первого render pass'а** создаём второй, тоже рисующий на экран, но **без буфера глубины**; `loadOp = Load`, `storeOp = Store`
- Выставляем созданную bind группу под индексом 0: `render_pass.set_bind_group(0, bind_group)`
- В коде уже есть `texturePipeline` – выставляем ей `TextureImmediates` и рисуем `render_pass.draw(vertex_count = 6)`, чтобы в левой нижней четверти экрана появился прямоугольник
- NB: Текстура будет чёрной – мы пока ничего в неё не рисовали

Задание 2



Задание 3

Рисуем в нашу текстуру

- Изменяем настройку **первого render pass'а**, чтобы он рисовал в нашу текстуру
- Придётся изменить формат пикселя при создании `mainRenderPipeline`
- Придётся изменить размер буфера глубины (до совпадающего с размером текстуры)

Задание 3



Задание 4

Создаём array-текстуру

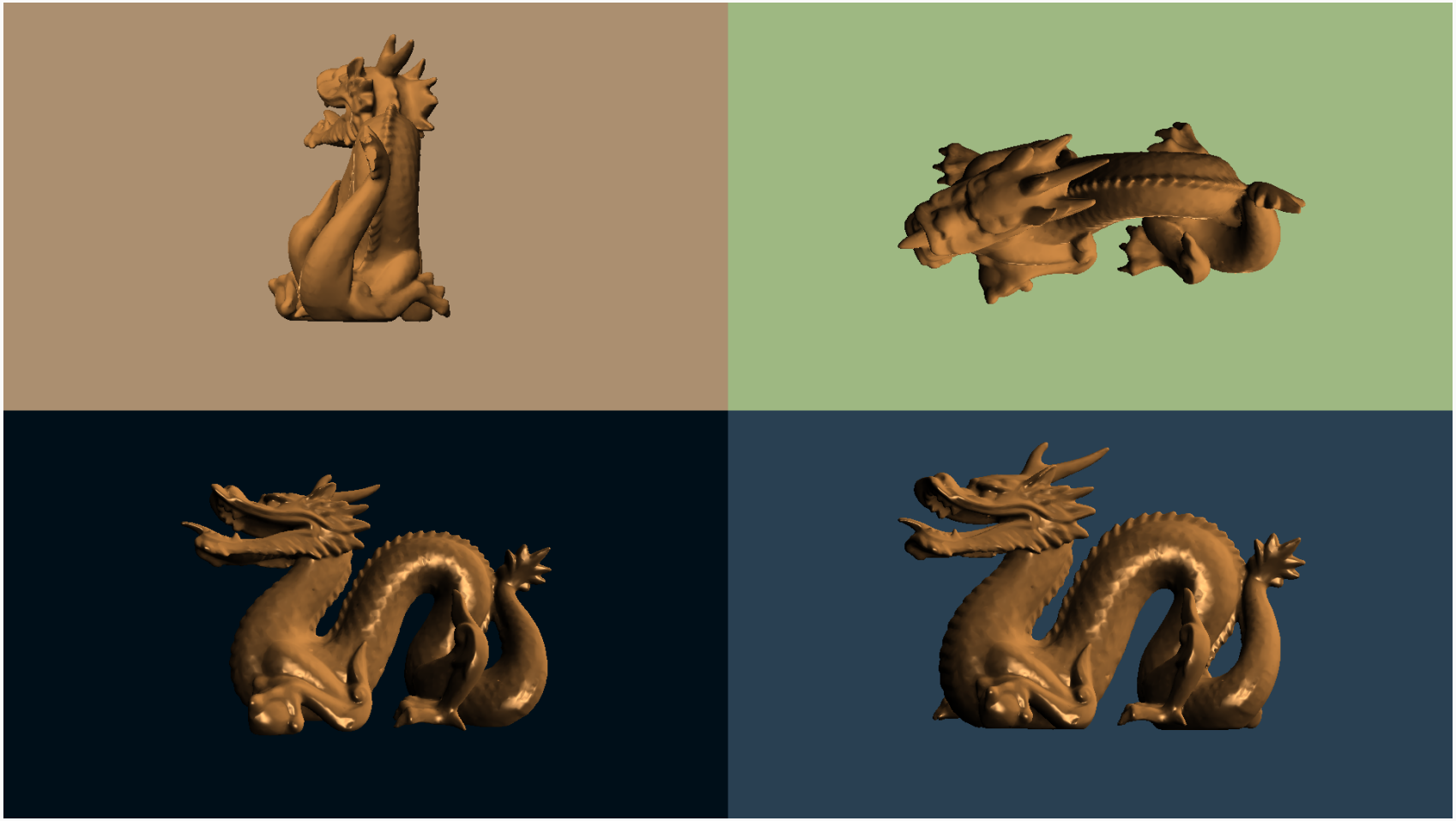
- При создании нашей текстуры указываем размер по третьей координате равный `z = 4`
- Меняем одну texture view и одну bind группу на массивы из 4х текстур и bind групп, соответственно
- Texture view с номером *k* будет ссылаться на `k`-ый слой array текстуры, для этого при её создании нужно указать `base_array_layer = k`
- `k`-ая bind группа ссылается на `k`-ый texture view
- Для рендеринга пока используем `0`-ую bind группу и `0`-ой texture view
- NB: Пока ничего не изменится, мы никак не используем оставшиеся 3 слоя

Задание 5

Рисуем модель 4 раза в разные слои текстуры

- Заносим **первый render pass** в цикл: k-ая итерация рисует в k-ый слой текстуры (т.е. в k-ый texture view)
- У каждой итерации делаем свой цвет очистки и свою матрицу проекции (одна перспективная, три ортографических с разных сторон)
- Заносим рисование прямоугольника с текстурой на экран в цикл: k-ая итерация рисует k-ый слой (т.е. использует k-ую bind группу)
- Прямоугольники должны вместе покрыть экран
- NB: В сумме должно получиться **5 render pass'ов**

Задание 5



Задание 6

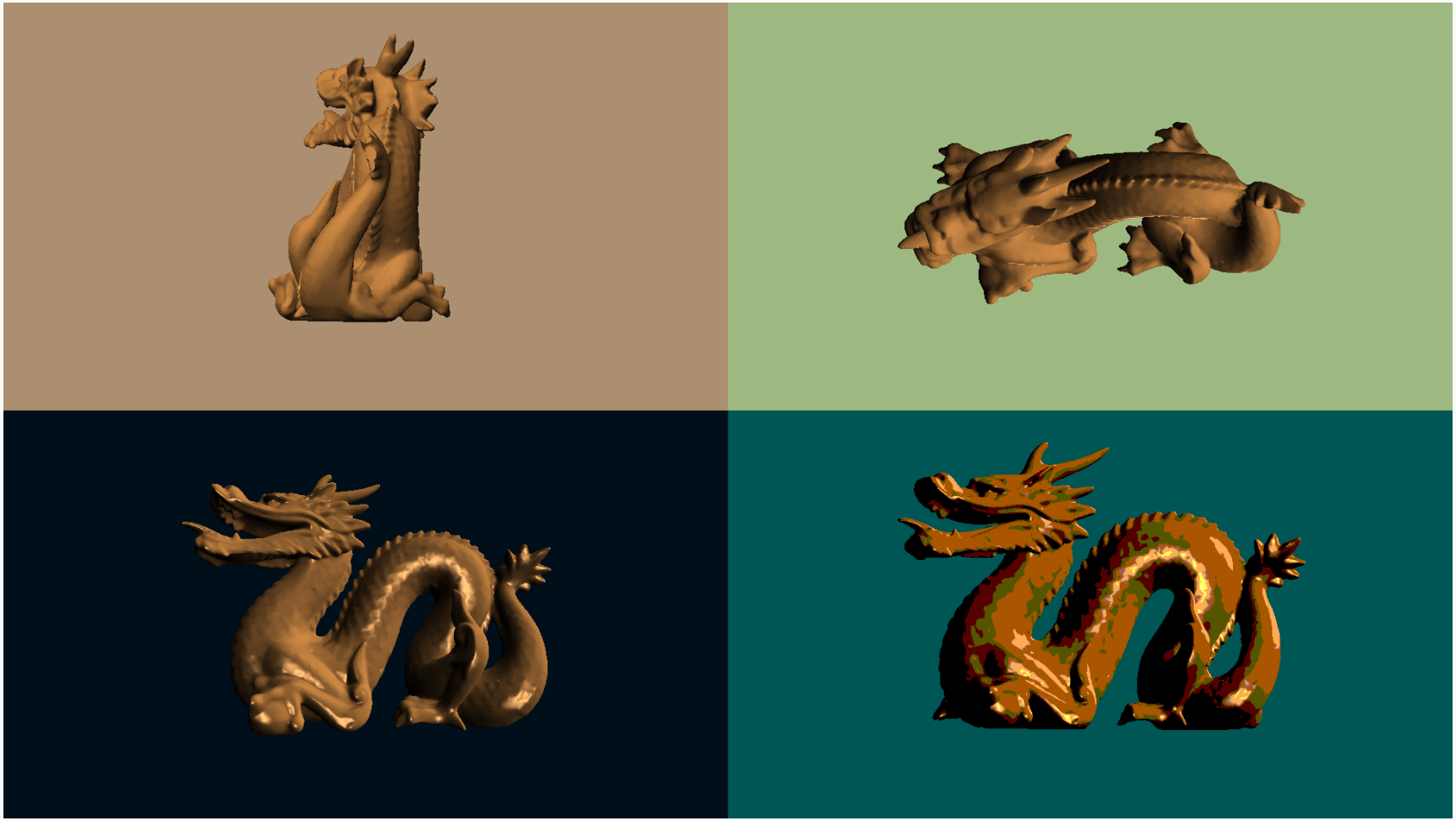
Добавляем color grading

- Добавляем в `TextureImmediates` поле `mode: i32` и устанавливаем его в номер текущей итерации
- В шейдере `shader_texture.wgsl` при `mode == 1` делаем color grading: преобразуем выходной цвет как

```
color = floor(color * 4.0) / 3.0;
```

- В противном случае возвращаем цвет без изменений

Задание 6



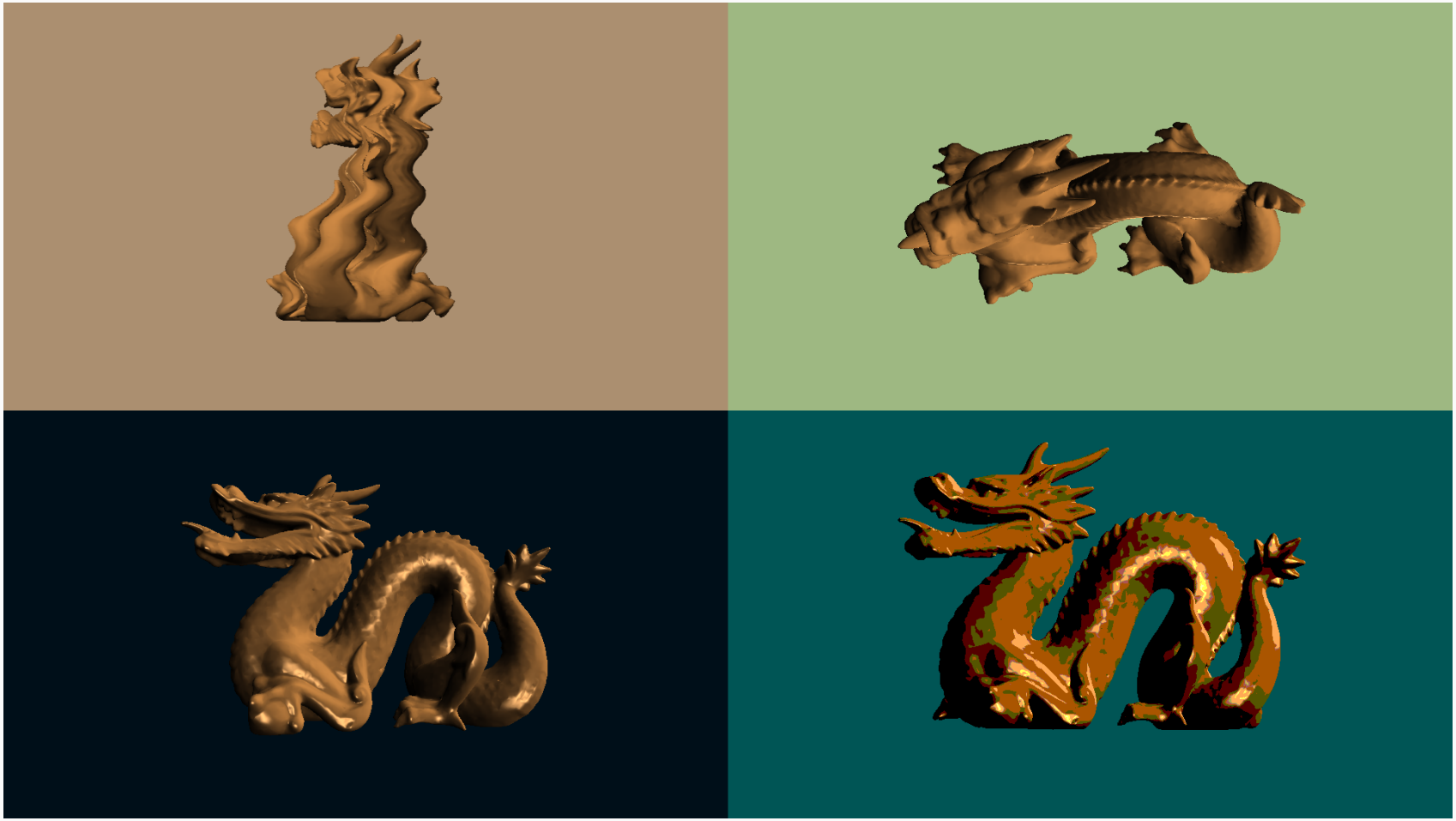
Задание 7

Добавляем domain warping

- Добавляем в `TextureImmediates` поле `time: f32` и передаём туда время (оно уже есть в заготовке)
- В шейдере `shader_texture.wgsl` при `mode == 2` делаем domain warping: перед чтением из текстуры искажаем текстурные координаты чем-нибудь зависящим от времени, например

```
vec2(sin(in.texcoord.y * 50.0 + immediates.time) * 0.01, 0.0)
```

Задание 7



Задание 8*

Добавляем гауссово размытие

- В шейдере `shader_texture.wgsl` при `mode == 3` делаем гауссово размытие, например, с `N = 10` и `sigma = 6`
- Если будет тормозить, можно взять значения поменьше
- Код шейдера можно взять из слайдов лекции

Задание 8*

